Nama : Fadiyah Nurilwalid Hajar

NRP : 3125522012

Halaman 1 : Informasi Proyek & Sprint

- **Nama Proyek:** `P7_3125522012_FadiyahNurilwalidHajar` (sesuaikan dengan NRP dan nama Anda)
- **Sprint Goal:** Mengembangkan desain proyek P6 dengan menerapkan *abstract class* dan *interface* yang sesuai agar struktur class lebih jelas, perilaku objek memiliki kontrak yang konsisten, dan program tetap dapat berjalan.
- Sprint Backlog (7/7 Selesai):
  - [x] SB-01 Audit hierarchy dan behavior proyek P6
  - [x] SB-02 Menentukan kandidat abstract class (`Person`)
  - [x] SB-03 Menentukan abstract method (`tampilkanInfo`)
  - [x] SB-04 Menentukan kandidat interface (`DapatDicetak`)
  - [x] SB-05 Memperbarui class diagram
  - [x] SB-06 Mengimplementasikan abstract class dan interface
  - [x] SB-07 Menguji polymorphism dan program
- Tabel Audit Desain P6:

Class / Behavior	Kondisi P6	Rencana P7	Alasan
Person (atau Superclass Entitas)	Class biasa	Abstract class	Hanya mewakili konsep umum entitas manusia (pelanggan/pegawai) dan tidak layak diinstansiasi secara langsung.
prosesTransaksi() / **hitung()**	Method biasa	Abstract method	Setiap subclass konkret (misal: pembayaran tunai/nontunai atau jenis transaksi) memiliki aturan dan cara hitung yang spesifik.
DapatDicetak	Belum ada	Interface	

Halaman 2: Desain & Penjelasan Kode

- **Class Diagram Terbaru:** Buat skema diagram UML yang menunjukkan hubungan *Generalization* (`extends` dari Abstract Class ke Subclass) dan *Realization* (`implements` dari Interface ke Class).
- Penjelasan Komponen:
  - **Abstract Class (`Person`):** Berisi atribut umum (`nama`, `alamat`), constructor, dan `abstract void tampilkanInfo();`.

- **Subclass Konkret (`Pelanggan` & `PegawaiToko`):** Mengimplementasikan *abstract method* secara spesifik.
- **Interface (`DapatDicetak`):** Mendefinisikan kontrak method `cetakStruk()`.

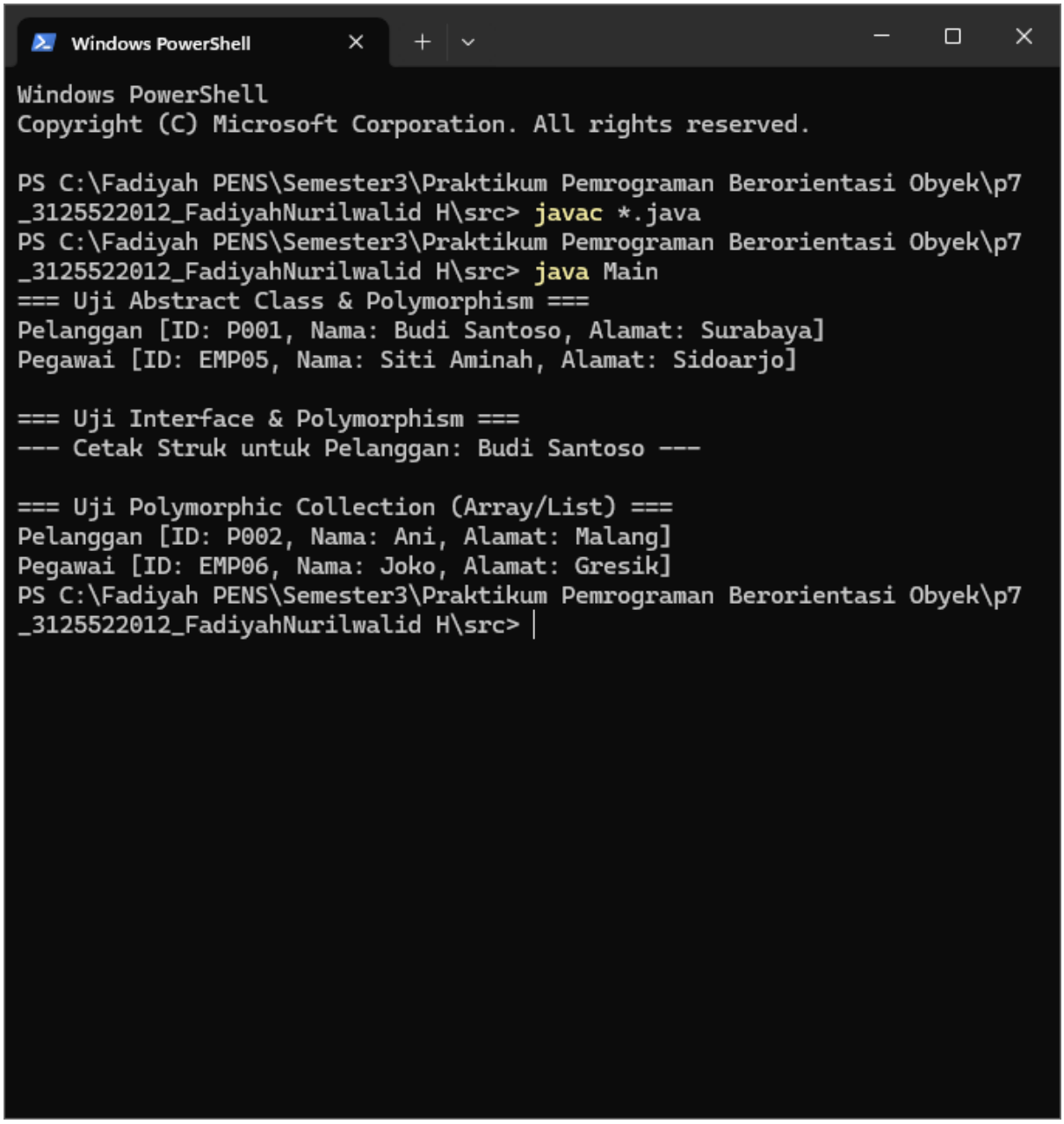
- **Alasan Perubahan Desain:** Menerapkan prinsip *abstraction* agar struktur kode lebih rapi, berbasis kontrak perilaku, dan mendukung *polymorphism* yang lebih aman.

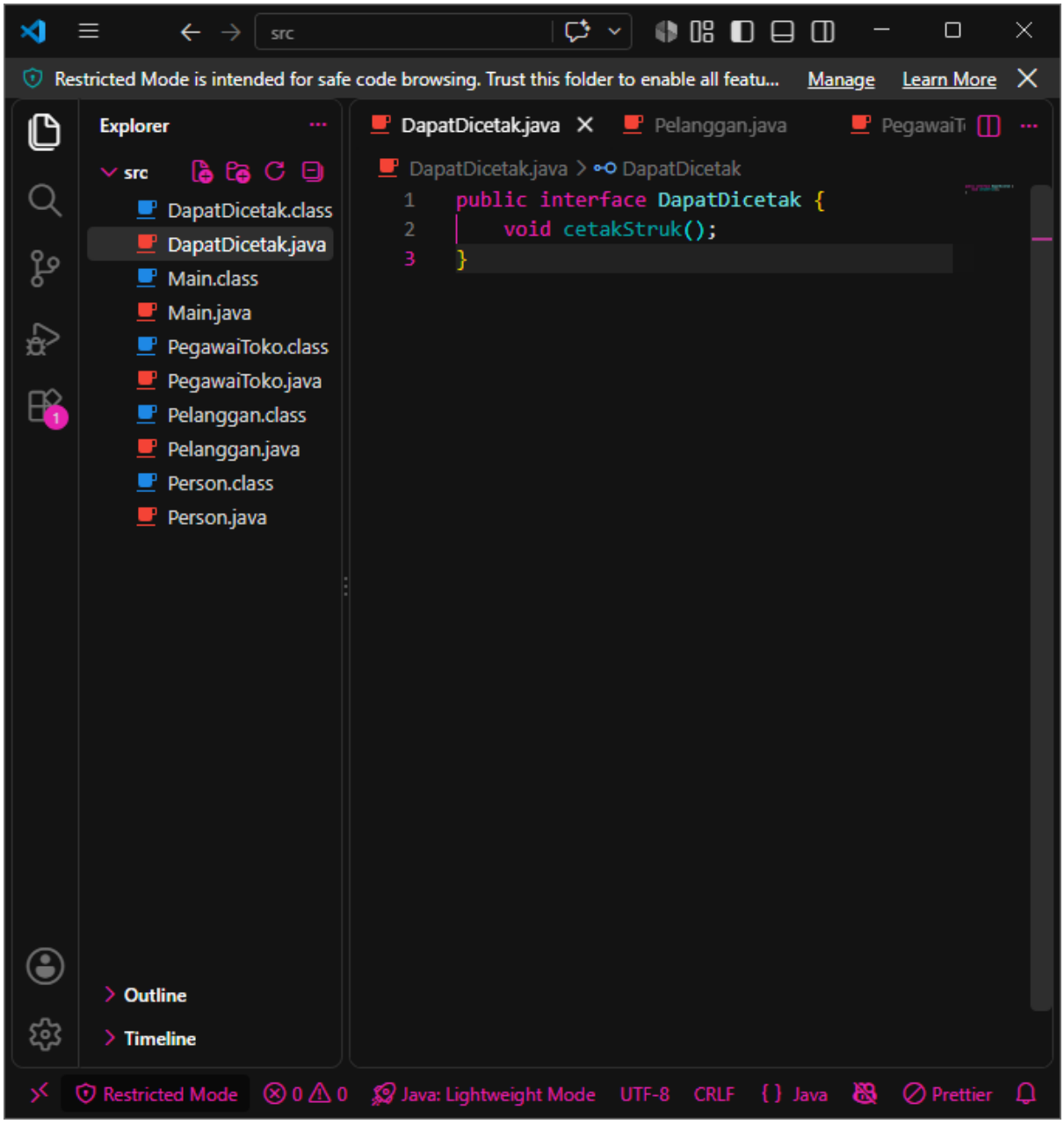
Halaman 3: Pengujian, Sprint Review, & Retrospective

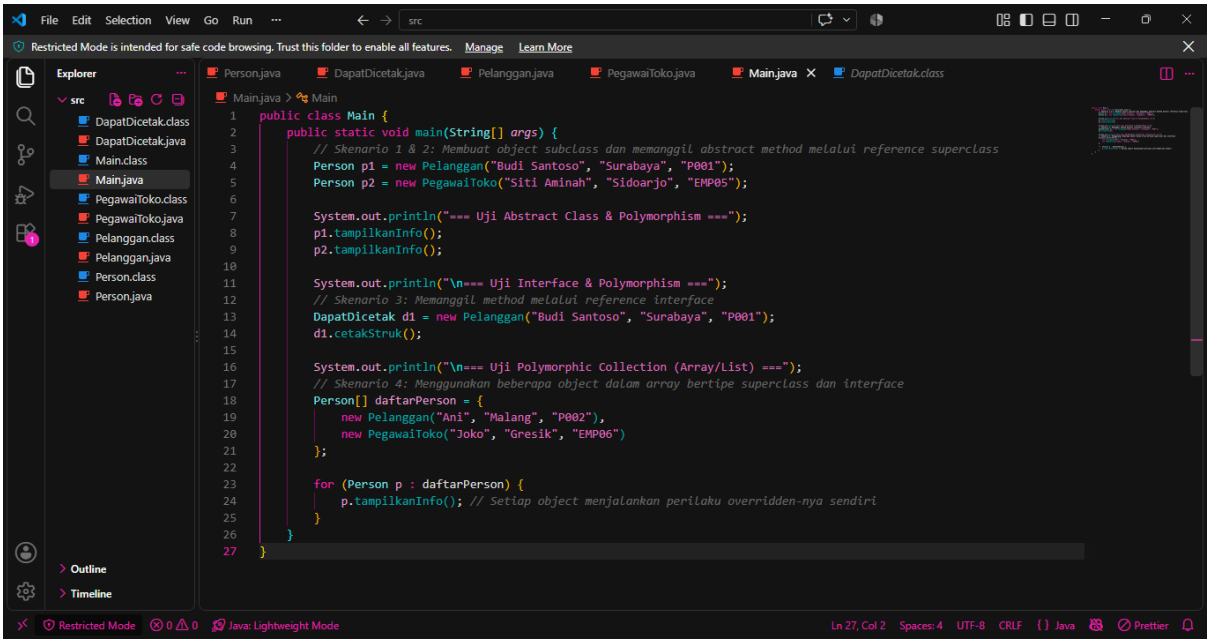
- Tabel Skenario Pengujian:

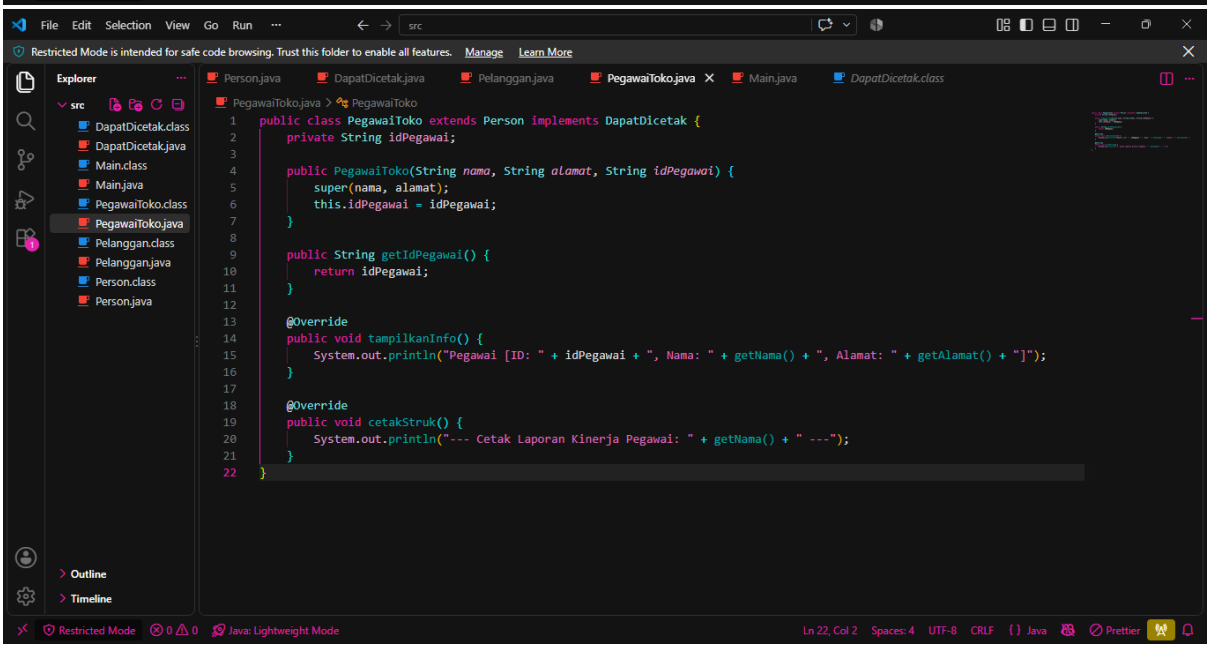
No	Skenario	Hasil yang Diharapkan
1	Membuat object subclass	Object berhasil dibuat
2	Memanggil abstract method lewat reference superclass	Method subclass yang sesuai dijalankan
3	Memanggil method melalui reference interface	Implementasi interface yang sesuai dijalankan
4	Menggunakan beberapa object dalam array bertipe superclass	Setiap object menjalankan perilakunya sendiri

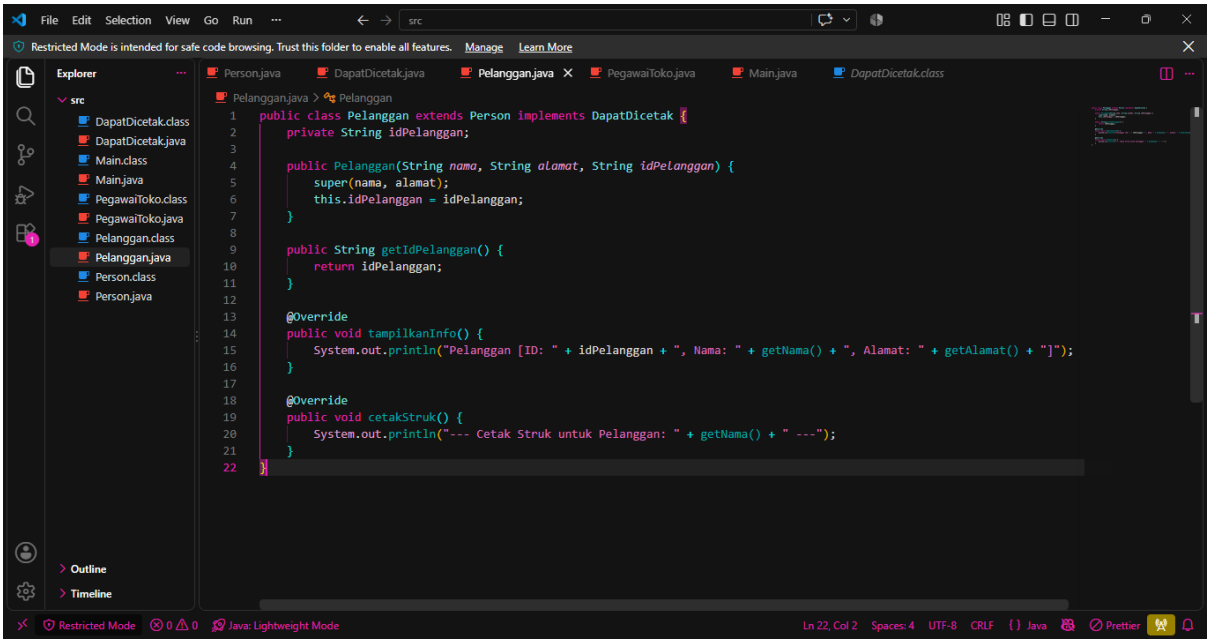
- Screenshot Hasil Running:

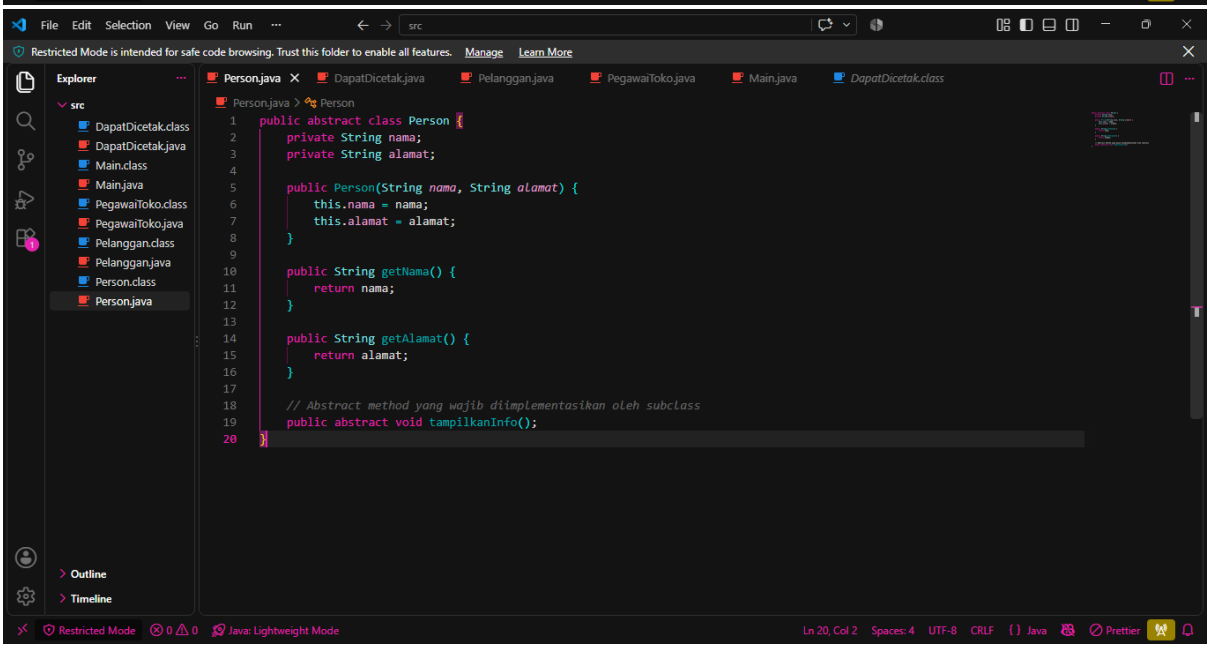

Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
PS C:\Fadiyah PENS\Semester3\Praktikum Pemrograman Berorientasi Obyek\p7_3125522012_FadiyahNurilwalid H\src> javac *.java
PS C:\Fadiyah PENS\Semester3\Praktikum Pemrograman Berorientasi Obyek\p7_3125522012_FadiyahNurilwalid H\src> java Main
=== Uji Abstract Class & Polymorphism ===
Pelanggan [ID: P001, Nama: Budi Santoso, Alamat: Surabaya]
Pegawai [ID: EMP05, Nama: Siti Aminah, Alamat: Sidoarjo]
=== Uji Interface & Polymorphism ===
--- Cetak Struk untuk Pelanggan: Budi Santoso ---
=== Uji Polymorphic Collection (Array/List) ===
Pelanggan [ID: P002, Nama: Ani, Alamat: Malang]
Pegawai [ID: EMP06, Nama: Joko, Alamat: Gresik]
PS C:\Fadiyah PENS\Semester3\Praktikum Pemrograman Berorientasi Obyek\p7_3125522012_FadiyahNurilwalid H\src>


src
Restricted Mode is intended for safe code browsing. Trust this folder to enable all featu...
Manage
Learn More
Explorer
src
DapatDicetak.class
DapatDicetak.java
Main.class
Main.java
PegawaiToko.class
PegawaiToko.java
Pelanggan.class
Pelanggan.java
Person.class
Person.java
Outline
Timeline
DapatDicetak.java
Pelanggan.java
DapatDicetak.java > DapatDicetak
1 public interface DapatDicetak {
2 void cetakStruk();
3 }
Restricted Mode
0 0
Java: Lightweight Mode
UTF-8
CRLF
Java
Prettier


File Edit Selection View Go Run
src
Restricted Mode is intended for safe code browsing. Trust this folder to enable all features. Manage Learn More
Explorer
src
DapatDicetak.class
DapatDicetak.java
Main.class
Main.java
PegawaiToko.class
PegawaiToko.java
Pelanggan.class
Pelanggan.java
Person.class
Person.java
Person.java DapatDicetak.java Pelanggan.java PegawaiToko.java Main.java DapatDicetak.class
Main.java > Main
public class Main {
public static void main(String[] args) {
// Skenario 1 & 2: Membuat object subclass dan memanggil abstract method melalui reference superclass
Person p1 = new Pelanggan("Budi Santoso", "Surabaya", "P001");
Person p2 = new PegawaiToko("Siti Aminah", "Sidoarjo", "EMP05");
System.out.println("=== Uji Abstract Class & Polymorphism ===");
p1.tampilkanInfo();
p2.tampilkanInfo();
System.out.println("\n=== Uji Interface & Polymorphism ===");
// Skenario 3: Memanggil method melalui reference interface
DapatDicetak d1 = new Pelanggan("Budi Santoso", "Surabaya", "P001");
d1.cetakStruk();
System.out.println("\n=== Uji Polymorphic Collection (Array/List) ===");
// Skenario 4: Menggunakan beberapa object dalam array bertipe superclass dan interface
Person[] daftarPerson = {
new Pelanggan("Ani", "Malang", "P002"),
new PegawaiToko("Joko", "Gresik", "EMP06")
};
for (Person p : daftarPerson) {
p.tampilkanInfo(); // Setiap object menjalankan perilaku overridden-nya sendiri
}
}
}
Outline
Timeline
Restricted Mode 0 0 Java: Lightweight Mode
Ln 27, Col 2 Spaces: 4 UTF-8 CRLF { } Java Prettier


File Edit Selection View Go Run
src
Restricted Mode is intended for safe code browsing. Trust this folder to enable all features. Manage Learn More
Explorer
src
DapatDicetak.class
DapatDicetak.java
Main.class
Main.java
PegawaiToko.class
PegawaiToko.java
Pelanggan.class
Pelanggan.java
Person.class
Person.java
Person.java DapatDicetak.java Pelanggan.java PegawaiToko.java Main.java DapatDicetak.class
PegawaiToko.java > PegawaiToko
public class PegawaiToko extends Person implements DapatDicetak {
private String idPegawai;
public PegawaiToko(String nama, String alamat, String idPegawai) {
super(nama, alamat);
this.idPegawai = idPegawai;
}
public String getIdPegawai() {
return idPegawai;
}
@Override
public void tampilkanInfo() {
System.out.println("Pegawai [ID: " + idPegawai + ", Nama: " + getNama() + ", Alamat: " + getAlamat() + "]");
}
@Override
public void cetakStruk() {
System.out.println("--- Cetak Laporan Kinerja Pegawai: " + getNama() + " ---");
}
}
Outline
Timeline
Restricted Mode 0 0 Java: Lightweight Mode
Ln 22, Col 2 Spaces: 4 UTF-8 CRLF { } Java Prettier





Sprint Retrospective

- What Went Well?
  - Implementasi *abstract class* `Person` berhasil merangkum atribut dan perilaku umum yang diwariskan dengan baik ke *subclass* konkrit (`Pelanggan` dan `PegawaiToko`), serta penerapan *interface* `DapatDicetak` sukses menyediakan kontrak perilaku lintas hierarki tanpa kendala kompilasi.
- What Went Wrong?
  - Sedikit kendala dalam menentukan pemisahan tanggung jawab secara tepat antara method yang harus dibuat *abstract* murni di *superclass* dengan method yang dapat diimplementasikan langsung.

- Improvement
  - Pada sprint berikutnya, perancangan struktur relasi antar class pada diagram UML akan dilakukan lebih teliti dan mendalam sebelum memulai penulisan kode program.